

O que é CSS

Quando iniciamos o processo de aprendizado no contexto de Programação Web Front-end, HTML é a primeira linguagem da qual temos contato, uma vez que é via HTML que inserimos o conteúdo e definimos a estrutura de uma página Web. O passo seguinte nesse processo de aprendizado é atuar com CSS, para que possamos encontrar meios para adicionar estilo à página Web desenvolvida.

A primeira versão do CSS aprovada pela W3C foi criada em 1996. Atualmente, a terceira e mais nova versão é a CSS3, que foi iniciada em 2012 e vem recebendo atualizações constantes. Você pode conferir os detalhes de cada versão diretamente pelo link da W3C - <https://www.w3.org/Style/CSS/#specs>

Cascading Style Sheets (CSS), ou ainda Folhas de Estilo em Cascata, é um mecanismo usado para estilizar e tornar as páginas Web personalizadas. Ou seja, a partir do CSS, você poderá:

- Descrever como os documentos devem ser apresentados, permitindo realizar ações como alterar a fonte, a cor e o tamanho e o espaçamento do seu conteúdo; adicionar animações e adicionar efeitos decorativos;
- Descrever regras de como você deseja estilizar o conteúdo de uma página Web;
- Propor um padrão nas escolhas de estilo em um determinado projeto.

Dentro desse contexto, podemos destacar as seguintes vantagens em utilizar CSS:

- O uso de CSS separa o conteúdo da apresentação;
- Criadores de conteúdo não precisam se preocupar com a apresentação;
- Possibilidade de reuso de dados;
- Possibilidade de customizar o estilo de acordo com as preferências e as necessidades do usuário;
- Uma única folha de estilo pode definir a consistência de uma coleção de documentos.

A sintaxe do CSS

Basicamente, o código CSS é definido por um conjunto de seletores.

Um seletor representa uma estrutura usada como uma condição, que determina quais e de que forma os elementos HTML serão estilizados. A sintaxe básica de um seletor é:

```
seletor {  
  propriedade: valor;  
}
```

- **seletor**: representa uma estrutura, utilizada como uma condição para determinar quais elementos de um grupo serão formatados. Exemplo: H1, p, body, ...
- **propriedade**: representa a característica que você deseja modificar no elemento. Exemplo: fonte, cor, tamanho, ...
- **valor**: representa valor referente a esta característica. Exemplo: Arial, red, 12px, ...

Por exemplo:

```
body {  
  color: red;  
}
```

declaração CSS

```
p {  
  font-family: arial;  
  text-align: center;  
}
```

bloco de declaração

```
h1 {  
  background: blue;  
}
```

conjunto de regras para o seletor p

O valor associado a uma propriedade (exemplo: a cor vermelha ao atributo *color*), é chamado de **declaração CSS**. Veja que as declarações CSS são colocadas em blocos de declaração, que por sua vez, são pareados com os seletores, produzindo conjuntos de **regras CSS**. No exemplo anterior, o conjunto de regras CSS do seletor p possui duas declarações.

Em CSS, para aplicar o estilo em uma página Web, o navegador é responsável por converter as regras CSS em um mapa de estilos **CSS Object Model (CSSOM)**, ou Modelo de Objeto CSS, da mesma forma que em HTML existe o modelo DOM.

A partir desse mapa, ele passa por cada regra definida no CSS, criando uma árvore de nós, com base nos seletores CSS que foram especificados. Ou seja, ocorre um processo similar ao objeto DOM, para renderizar o conteúdo HTML. É a partir do CSSOM também que temos a possibilidade de manipular o CSS por meio do JavaScript, como veremos mais adiante na disciplina.

Definir quais propriedades você deverá utilizar e quais valores atribuir a elas é o elemento central no enquanto,

Por exemplo, estas são algumas das mais comumente utilizadas:

- **background**: propriedade abreviada para definir os valores da cor de fundo em um elemento na folha de estilo.
- **background-image**: define uma imagem de fundo.
- **color**: define o valor da cor de um elemento com seu conteúdo em texto.
- **font-size**: define o tamanho da fonte.
- **font-family**: define lista de prioridades de famílias de fontes e/ou nomes genéricos de famílias de fontes.
- **font-style**: define propriedades de estilos (normal, italic ou oblique).
- **text-align**: controla o posicionamento horizontal do conteúdo de um elemento (left, right, center, justify).
- **text-decoration**: define efeito decorativo em um texto. Exemplos: underline (sublinhado); line-through (linha cortando o texto); e blink (efeito piscante).
- **text-transform**: controla os efeitos de capitalização do texto. Exemplos: capitalize, uppercase, lowercase.
- **width**: determina a largura da área de conteúdo de um elemento.

Você pode encontrar a documentação contendo uma lista completa das propriedades por meio deste link - <https://developer.mozilla.org/en-US/docs/Web/CSS/Reference>

Para inserir um comentário em um arquivo CSS, utiliza-se o /* e */ :

/*inserindo um comentário em arquivo CSS*/

```
h1 {  
    background: blue; /*inserindo outro comentário*/ }
```

Tipos de seletores CSS

Como foi visto no passo anterior, um seletor é responsável por definir o estilo de um ou mais elementos HTML. Para isso, os seletores mais utilizados são os seguintes:

1. Seletor por elemento HTML;
2. Seletor por ID;
3. Seletor por classe;
4. Seletor por atributo;
5. Seletor por pseudo-classe;
6. Seletor universal.

Seletor por elemento HTML

Aplica o estilo aos elementos pelo nome do elemento HTML (exemplo: p, body, div, H1, img). Ele seleciona e aplica a regra CSS a todos os elementos do tipo fornecido em um documento.

Sintaxe: elemento { propriedades de estilo }

No exemplo a seguir, será alterada a cor da fonte (propriedade color) para vermelho, de todos os parágrafos (tag <p>). Também será inserida a cor amarelo como cor de fundo (propriedade background) de toda a página (tag <body>).

CSS	HTML	resultado
<pre>p { color: red; } body { background: yellow; }</pre>	<pre><body> <p> Aula de CSS </p> </body></pre>	

Seletor por ID

Uma das coisas que precisamos recapitular sobre HTML é que os elementos podem ter atributos. Um desses atributos refere-se ao id, que deve ser único para cada elemento. Ou seja, esse atributo especifica um **ID exclusivo** para um elemento HTML, portanto, seu valor deve ser exclusivo no documento.

Nesse contexto, utilizamos o seletor por ID em casos nos quais há apenas um elemento HTML com um determinado valor no atributo ID e desejamos aplicar a regra de estilo somente a esse elemento.

Sintaxe: `#nome_id { propriedades de estilo }`

Veja que o seletor deve ser precedido por uma hashtag (#) e seu nome deve ser exatamente o mesmo que estiver descrito no atributo ID de um elemento HTML, conforme o exemplo a seguir.

CSS	HTML	resultado
<pre>#t1 { color: red; }</pre> <pre>#t2 { color: orange; font-style:italic; }</pre>	<pre><h1 id="t1">Aula de CSS</h1></pre> <pre><h2 id="t2">tipos de seletores</h2></pre>	Aula de CSS <i>tipos de seletores</i>

Seletor por classe

Além do atributo ID, um elemento HTML também pode possuir um atributo chamado `class`, que corresponde a uma classe do elemento. Atenção: diferente do ID, um valor de atributo `class` pode ser aplicado a mais elementos.

Dessa forma, o seletor por classe aplica as regras CSS aos elementos HTML que possuam o atributo `class` especificado.

Sintaxe: `.nome_classe { propriedades de estilo }`

Perceba que nesse seletor, o nome da classe deve ser precedido pelo ponto, conforme exemplo a seguir.

CSS	HTML	resultado
<pre>.destaque { color: red; font-size:200%; }</pre> <pre>.aviso { background: yellow; }</pre>	<pre><p class="destaque">Aula de CSS</p></pre> <pre><p class="aviso">Dia de avaliação</p></pre> <pre><p class="aviso">Revise o conteúdo</p></pre>	Aula de CSS Dia de avaliação Revise o conteúdo

Seletor por atributo

Vimos que os elementos HTML possuem diferentes tipos de atributos. Por exemplo, o elemento <a> possui o atributo href e o elemento possui o atributo src. Em casos nos quais desejamos aplicar a regra CSS somente quando um atributo de um código HTML tiver um valor específico, utilizamos esse tipo de seletor.

Sintaxe: elemento[atributo=valor] { propriedades de estilo }

Veja um exemplo a seguir, no qual é aplicado uma regra CSS que torna a cor de fundo amarelo somente ao elemento <a> cujo atributo href tenha o valor “https://www.css.com”.

CSS	HTML	resultado
<pre>a { color: green; } a[href="http://www.css.com"] { background-color: yellow; }</pre>	<pre> Link da aula Link especial </pre>	• Link da aula • Link especial

Seletor por pseudo-classe

Uma pseudo-classe é usada para definir um estado especial de um elemento. Um exemplo desse estado pode ser identificado em links (<a>), que podem ter um estado quando o link é visitado ou quando se passa o mouse sobre ele.

Dessa forma, esse seletor é utilizado indicando o nome do elemento, seguido de dois pontos e uma palavra-chave que especifica um estado especial do elemento HTML selecionado.

Sintaxe: `seletor:pseudo-classe { propriedade: valor }`

Veja um exemplo a seguir, no qual, ao clicar com o mouse sobre o link (estado hover), a cor do texto é alterada para vermelho.

CSS	HTML	resultado
<pre>a { color: green; } a:hover { color: red; }</pre>	<pre> Link da aula Link especial </pre>	• Link da aula • Link especial

Seletor universal

Por fim, utiliza-se o seletor universal quando se deseja aplicar uma regra CSS a todos os elementos de uma página. Para isso, o asterisco (*) é utilizado como nome do seletor.

Sintaxe: `* { propriedades de estilo }`

Veja um exemplo a seguir, no qual é aplicada a cor de fonte verde para todos os elementos da página.

CSS	HTML	resultado
<pre>* { color: green; }</pre>	<pre><h1>Aula de CSS</h1> <p>Seletor universal</p></pre>	Aula de CSS Seletor universal

Além de saber como utilizar os diferentes tipos de seletores, é importante saber que também é possível realizar o agrupamento de seletores, ou utilizá-los de forma hierárquica.

Ou seja, ao invés de indicar a regra CSS em cada seletor, você pode agrupá-los, tornando seu código mais limpo. Veja alguns exemplos:

Com agrupamento: Os estilos serão aplicados a todos os <h1>, <h2> e <p>

```
h1, h2, p {  
    color: blue;  
    font-family: Arial, sans-serif;  
}
```

Sem agrupamento: Os estilos serão aplicados a todos os <h1>, <h2> e <p>

```
h1 {  
    color: blue;  
}
```

```
h2 {  
    color: blue;  
}
```

```
p {  
    color: blue;  
}
```

Aplica o mesmo estilo para:

- elementos com classe .button
- elementos com classe .link
- elemento com id #menu-item

```
.button, .link, #menu-item {  
    background-color: black;  
    color: white;  
    padding: 10px;  
}
```

Importante:

- Separe os seletores com **vírgula (,)**
- O estilo será aplicado a **todos os seletores listados**

Quando você escreve **sem vírgula**, como:

```
div p {  
  color: red;  
}
```

Isso **não é agrupamento**. Aqui você está usando um **seletor descendente**, que significa: "aplique o estilo apenas aos <p> que estão dentro de um <div>"

Compare:

Com vírgula (agrupamento)		Sem vírgula (descendente)	
div, p { color: red; }	Aplica a todos os <div> Aplica a todos os <p>	div p { color: red; }	Aplica somente aos <p> que estão dentro de <div>

Exemplo HTML:

```
<div>  
  <p>qual a cor deste texto, p dentro de div</p>  
</div>  
<p>qual a cor deste texto, p fora de div</p>
```

- Se utilizarmos o CSS com agrupamento, o texto das duas tags <p> ficará em vermelho
- Se utilizarmos o CSS com descendente, apenas o texto da tag <p> que está dentro da tag <div> ficará vermelho

Importante:

- $a, b \rightarrow$ **agrupamento** ($a \underline{\in} b$)
- $a b \rightarrow$ **hierarquia** (b dentro de a)

1. Descendente (A B)

```
div p {  
  color: red;  
}
```

Seleciona **todo <p> dentro de <div>**, em qualquer nível.

```
<div>  
  <section>  
    <p>Esse é afetado</p>  
  </section>  
</div>
```

2. Filho direto (A > B)

```
div > p {  
  color: blue;  
}
```

Seleciona apenas <p> que são **filhos diretos** de <div> (um nível só).

```
<div>  
  <p>Esse é afetado</p>  
  <section>  
    <p>Esse NÃO é afetado</p>  
  </section>  
</div>
```

3. Irmão adjacente (A + B)

```
h1 + p {  
  color: green;  
}
```

Seleciona o <p> que vem **imediatamente depois** de um <h1>.

```
<h1>Título</h1>  
<p>Esse é afetado</p>  
<p>Esse NÃO é afetado</p>
```

4. Irmãos gerais (A ~ B)

```
h1 ~ p {  
  color: purple;  
}
```

Selecione **todos os <p> que vêm depois de um <h1>**, no mesmo nível.

```
<h1>Título</h1>  
<p>Esse é afetado</p>  
<p>Esse também é afetado</p>
```

Importante

- $a \rightarrow b$ → qualquer descendente
- $a < b$ → filho direto
- $a + b$ → próximo irmão (logo depois)
- $a \sim b$ → todos os irmãos seguintes

Exemplo:

Dado o HTML:

```
<div>  
  <p id="p1">Parágrafo 1</p>  
  <section>  
    <p id="p2">Parágrafo 2</p>  
  </section>  
  <p id="p3">Parágrafo 3</p>  
</div>
```

```
<p id="p4">Parágrafo 4</p>
```

```
<h1>Título</h1>  
<p id="p5">Parágrafo 5</p>  
<p id="p6">Parágrafo 6</p>
```

1 – Qual ou quais parágrafos ficam **vermelho** com este CSS:

```
div p {  
    color: red;  
}
```

2 – Qual ou quais parágrafos ficam **azul** com este CSS:

```
div > p {  
    color: blue;  
}
```

3 – Qual ou quais parágrafos ficam **verde** com este CSS:

```
h1 + p {  
    color: green;  
}
```

4 – Qual ou quais parágrafos ficam **roxo** com este CSS:

```
h1 ~ p {  
    color: purple;  
}
```

Respostas

1 – div p Selecione todos dentro de div: • p1 ☒ • p2 ☒ • p3 ☒	3 – h1 + p Apenas o próximo após <h1>: • p5 ☒
2 – div > p Apenas filhos diretos: • p1 ☒ • p3 ☒ • p2 ☒ (porque está dentro de <section>)	4 – h1 ~ p Todos os <p> depois do <h1>: • p5 ☒ • p6 ☒

Formas de inclusão de CSS

Além dos tipos de seletores, também é importante que você saiba que existem **3 formas** de inserir o código CSS em um documento:

1. Folha de estilo externa
2. Folha de estilo interna
3. Estilo inline

Folha de estilo externa

Considerado atualmente o caso mais comum, refere-se à quando você define seu CSS escrito em um arquivo separado, com extensão .css, no qual você o refere dentro de um elemento `<link>` do documento HTML, conforme exemplo a seguir:

style.css	index.html
<pre>h1 { color: blue; background: yellow; } p { color: red; }</pre>	<pre><!DOCTYPE html> <html> <head> <meta charset="utf-8" /> <title>Exemplo</title> <link rel="stylesheet" href="css/style.css" /> </head> <body> <h1>Aula de CSS!</h1> <p>Criando CSS externo</p> </body> </html></pre>

O atributo href do elemento `<link>` precisa fazer referência a um arquivo do seu projeto. Nesse cenário, o arquivo estava dentro da pasta de nome css.

Folha de estilo interna

Nesse caso, o código fica no mesmo arquivo que o documento HTML. Para isso, você deve colocar o código CSS dentro de elemento `<style>`, localizado no elemento `<head>` do documento HTML, conforme exemplo a seguir, destacado em vermelho:

index.html
<pre><!DOCTYPE html> <html> <head> <meta charset="utf-8" /> <title>Exemplo</title> <style> h1 { color: blue; background: yellow; } p { color: red; } </style> </head> <body> <h1>Aula de CSS!</h1> <p>Criando CSS externo</p> </body> </html></pre>

Embora essa opção seja útil em alguns cenários, como quando você não tem acesso a edição de arquivos `.css`, a melhor opção ainda é a de uso de CSS externo, dado que o CSS pode ser atualizado em vários arquivos HTML sempre que alguma alteração for necessária.

Estilo inline

Por fim, uma outra opção menos indicada é a de CSS inline, sendo essas declarações CSS que afetam apenas um determinado elemento, de forma que o conteúdo CSS ocorre por um atributo `style`, conforme o exemplo a seguir.

index.html
<pre><!DOCTYPE html> <html> <head> <meta charset="utf-8" /> <title>Exemplo</title> </head> <body> <h1 style="color: blue; background: yellow;">Aula de CSS!</h1> <p style="color:red;">Criando CSS externo</p> </body> </html></pre>

“Não utilize essa opção, a menos que seja estritamente necessária!”.

Das 3 opções apresentadas, essa é a que mais dificulta a manutenção do código, dado que as regras de estilo estão espalhadas ao longo do documento HTML, dificultando sua compreensão.

Se for utilizado um arquivo HTML a inclusão de CSS externo, interno e inline, é importante saber que há uma ordem de prioridade na aplicação do estilo, que ocorre em cascata.

Além disso, caso diferentes estilos forem definidos para um mesmo elemento, utilizando o mesmo seletor, prevalece o de maior prioridade, conforme a ordem a seguir:

1. Estilo inline
2. Folha de estilo interna
3. Folha de estilo externa
4. Definição do navegador (browser)

Além disso, atenha-se as outras seguintes regras também:

- Caso haja a declaração tanto de folha de estilo externa quanto interna, a prioridade dependerá da ordem de declaração.
- Caso um seletor seja definido em mais de uma folha de estilo externa, a última referenciada terá a precedência.
- O navegador também selecionará a última opção de estilo, caso o seletor seja definido mais de uma vez.
- Para mudar essa ordem, utiliza-se a regra *!important* logo após o valor de uma propriedade, para sobrescrever as demais regras para o elemento.

Aprender a definir cores

Para definir o valor de uma propriedade referente a uma cor, é possível utilizar as seguintes opções:

- **Palavra-chave (em inglês):** nome predefinido da cor, como *black*, *white*, *blue*, *violet*, etc..
- **Escala RGB (vermelho, verde, azul):** define os valores de 0 a 255, para R (vermelho), G (verde) e B (azul). Exemplo: `rgb(0,0,0)` para preto e `rgb(255,255,255)` para branco.
- **Valores hexadecimais:** valores para R, G e B definidos com números hexadecimais (de 00 a FF, ou de 0 a 255). Exemplo: `#000000` para preto e `#FFFFFF` para branco.
- **Valores de saturação:** permite definir as cores com uso de parâmetros como saturação e luminosidade.

Você também consegue definir fatores como a transparência da cor e sua luminosidade, conforme o exemplo a seguir.

```
body {  
    background-color: yellow; /* Cor amarela*/  
    background-color: #409767 /*Pares hexadecimais RGB*/  
    background-color: rgb(255, 0, 0); /* Cor vermelha */  
    background-color: rgba(255, 0, 0, 0.7) /* Cor vermelha, com 70% de  
transparência */  
    background-color: hsl(15, 80%, 50%) /* (matiz (0-360), saturação,  
luminosidade) */  
    background-color: hsla(15, 80%, 50%, 0.5) /*(matiz(0-360), saturação,  
luminosidade, transparência) */  
}
```

Abra o arquivo **exemplo1.html** no navegador utilizando Live Server e visualize o estilo da página.

Perceba que o fundo da página está amarelo. Essa cor foi definida no arquivo *style.css*, que está na pasta css de exemplo da aula 05, conforme o código a seguir:

```
body{  
    background-color: yellow;  
}
```

Altere o arquivo, incluindo novas cores para os elementos header e footer, conforme exemplo a seguir:

```
body{
    background-color: green;
}

header{
    /*insira uma cor de fundo aqui*/
}

footer{
    /*insira uma cor de fundo aqui*/
}
```

Ao clicar na descrição da cor, é apresentada outras opções de formato, como hexadecimal, hsl e hwb.

Bem, é importante que você saiba que a propriedade background-color não é a única que podemos utilizar para definir uma cor. Podemos, por exemplo, usar a propriedade color para definir a cor de um elemento textual, ou para definir a cor de uma borda.

Para exemplificar, inclua o código a seguir no seu arquivo css:

```
h1{
    color: blue;
}
```

- ❖ Altere o arquivo style.css, incluindo uma nova cor para os elementos <p> e <h2>.
- ❖ Verifique se seu código CSS está adequado por meio do validador de CSS da W3C: <https://jigsaw.w3.org/css-validator/>

É importante lembrarmos também que, além da questão de aparência, muitas pessoas possuem dificuldade em distinguir cores, portanto, por questões de **acessibilidade**, é altamente recomendado a escolha de cores que tenham um bom contraste.

Você pode encontrar mais informações sobre a escolha de cores nos links a seguir:

- Contrast Checker: ferramenta para analisar o contraste das cores:
<https://webaim.org/resources/linkcontrastchecker/?fcolor=595959&bcolor=FFFFFF>
- Teste de contraste de cores: tutorial para fazer o teste de contraste por meio das ferramentas do desenvolvedor que o navegador oferece
<https://learn.microsoft.com/pt-br/microsoft-edge/devtools/accessibility/color-picker>
- Color Picker: ferramenta de seleção de cores da W3School
https://www.w3schools.com/colors/colors_picker.asp
- CSS Gradient: oferece a opção de gerar imagens com a definição de cores gradientes
<https://cssgradient.io/>

Alterar as propriedades de texto

Uma outra habilidade que você deve adquirir ao utilizar CSS é a de estilizar texto. Isso inclui saber trabalhar com cores, tipos de fontes, tamanhos e efeitos decorativos do seu texto. É essa habilidade que você irá adquirir nessa etapa. Vamos lá?

Para isso, os seguintes tópicos serão abordados:

1. Fontes
2. Decoração
3. Alinhamento

Fontes

Após a definição de cores do texto, a fonte é uma propriedade comumente utilizada para acrescentar estilo à página Web.

Uma das propriedades utilizadas para definir a fonte é a **font-family**. Ela permite especificar uma lista de uma ou mais nomes de famílias de fontes (como "Times New Roman", "Courier", "Arial", etc.) e/ou nomes de famílias genéricas (como "serif", "sans-serif" e "cursive") para o elemento.

Você pode conferir uma lista de fontes e seu formato por este link:

https://www.w3.org/Style/Examples/007/fonts.pt_BR.html

Esse conjunto é definido para que, caso o navegador não suportar a primeira fonte, ele tentará a próxima fonte. Por isso, quando for declarar essa propriedade, separe cada nome por uma vírgula e sempre declare por último na lista o nome de fonte genérico, para ser utilizado somente em última opção.

Veja alguns exemplos a seguir:

```
font-family: Times, "Times New Roman", Georgia, serif;
```

```
font-family: Verdana, Arial, Helvetica, sans-serif;
```

```
font-family: "Lucida Console", Courier, monospace;
```

Veja a seguir outras opções de formatação de fonte disponíveis:

- font-size: estabelece o tamanho da fonte.
 - Exemplos:
 - font-size: small; /*nomes possíveis: xx-small, x-small, small, medium, large, x-large, xx-large*/
 - font-size: 12px; /*tamanho definido em pixels.*/
 - font-size: 0.8em; /*1em equivale a 100%, ou seja, ao tamanho padrão de fonte.*/
 - font-size: 80%; /*porcentagem relativa ao tamanho de fonte padrão do navegador do usuário*/

Observação: por questões de acessibilidade, geralmente é melhor usar valores relativos ao tamanho de fonte padrão do usuário.

- font-style: usada para definir o estilo da fonte, podendo receber o valor normal, italic e oblique.
 - Exemplos:
 - font-style: italic;
 - font-style: normal;
 - font-style: oblique;
- font-weight: utilizado para definir o peso ou a intensidade da fonte (exemplo: deixar a fonte em negrito)
 - Exemplo:
 - font-weight: bold;

Visualize o resultado no arquivo `exemplofonte.html`, abrindo o arquivo no navegador via Live Server.

Além das fontes apresentadas, você também pode incluir fontes externas em seu código CSS, como, por exemplo, as fontes do Google Fonts:

<https://fonts.google.com/>

Para acessar o modo de inclusão, selecione uma fonte e acesse a opção “*Get embed code*”

Após isso, utilize a tag link para indicar a referência da fonte. Exemplo:

```
<link  
href="https://fonts.googleapis.com/css2?family=Bubblegum+Sans&family=Markazi+Text:wght@400..700& display=swap" rel="stylesheet">
```

Use também o seletor de classe indicado pelo site do Google Fonts para aplicar em seu texto, conforme exemplo a seguir:

```
.bubblegum-sans-regular {  
font-family: "Bubblegum Sans", sans-serif;  
font-weight: 400;  
font-style: normal;  
}
```

Esse nome deverá ser aplicado a uma tag de texto de seu código HTML, conforme exemplo a seguir:

```
<h1 class="bubblegum-sans-regular">Testando a fonte do Google</h1>
```

O resultado será similar ao exemplo a seguir:

Testando a fonte do Google

Decoração

Além de controlar aspectos da fonte, o CSS também permite adicionar efeitos de decoração no texto (exemplo: sublinhado, sobrescrito). Isso é feito a partir da propriedade ***text-decoration***, que se refere a uma propriedade abreviada das propriedades *text-decoration-line* (obrigatória), *text-decoration-color* e *text-decoration-style*.

Confira alguns exemplos de decoração a seguir:

- `text-decoration: underline;`
- `text-decoration: overline blue;`
- `text-decoration: underline overline;`
- `text-decoration: overline;`
- `text-decoration: line-through;`
- `text-decoration: underline dotted;`
- `text-decoration: underline dotted red;`
- `text-decoration: red wavy underline;`
- `text-decoration-style: underline solid blue;`
- `text-decoration-style: underline double;`
- `text-decoration-style: underline dashed;`
- `text-decoration-style: underline wavy;`

Visualize o resultado no arquivo **`exemplotextdecoration.html`**, abrindo o arquivo no navegador via Live Server.

Além desta propriedade, podemos adicionar também uma sombra ao texto, a partir da propriedade ***text-shadow***.

Essa propriedade requer 3 medidas, podendo também ser adicionada uma cor:

- Medida 1: deslocamento da sombra para a direita (valor positivo) ou para a esquerda (valor negativo);
- Medida 2: deslocamento da sombra para baixo (valor positivo) ou para cima (valor negativo);
- Medida 3: raio para efeito *blur* na sombra;
- Cor: define a cor da sombra.

Essa propriedade aceita uma lista de sombras. Para isso, as informações de cada sombra devem ser separadas por vírgula.

Veja alguns exemplos:

- `text-shadow: 2px 2px 8px red;`
- `text-shadow: 0 0 3px red, 2px 2px 8px yellow;`
- `text-shadow: 10px 10px 3px green;`

Visualize o resultado no arquivo **exemploshadow.html**, abrindo o arquivo no navegador via Live Server.

Alinhamento

Por fim, podemos identificar que é possível também definir como o texto ficará horizontalmente alinhado. Isso é realizado pela propriedade ***text-align***, que pode receber os seguintes valores:

- Alinhado à esquerda: `text-align: left;`
- Alinhado à direita: `text-align: right;`
- Centralizado: `text-align: center;`
- Justificado: `text-align: justify;`
- Inicial: configura a propriedade padrão. `text-align: initial;`
- Herdado: herda a propriedade do elemento pai. `text-align: inherit;`

Outra propriedade útil é a ***text-indent***, que especifica o recuo da primeira linha em um bloco de texto. Veja alguns exemplos a seguir:

- `text-indent: 20px;`
- `text-indent: 30%;`

- ❖ Inclua o código a seguir em seu arquivo style.css, para visualizar algumas dessas propriedades aplicadas ao arquivo exemplo1.html.

```
body{
    background-color: yellow;
}
h1{
    color: blue;
    text-decoration: underline dotted;
}
footer{
    text-align: center;
}
a{
    font-size: 150%;
    font-family: Arial, Helvetica, sans-serif;
}
p{
    font-style: italic;
    font-size: 120%;
    font-family: Arial, Helvetica, sans-serif;
}
.desc{
    color: white;
    text-shadow: 0 0 4px black;
}
#sigla{
    font-weight: bold;
    text-decoration: underline overline blue;
}
#ods{
    color: rgb(16, 37, 197);
    text-decoration-style: double;
}
```